UNIVERSITÉ ABDELMALEK ESSAADI
École Nationale des Sciences Appliquées d'Al Hoceima
Département Mathématique et Informatique
Module : J2EE / Java EE
Filière : TDIA 2

RAPPORT DE TP – JAVA EE (MVC2)

Application Web de Gestion des Produits en Java EE – Migration vers l'architecture MVC2 avec Hibernate

TP N°5

Encadré par : Pr. Mohamed Cherradi
Réalisé par : Essouiri Aya

Année universitaire : 2025 / 2026

Table des matières

1 Introduction **2**

2 Objectifs du TP5 **2**

3 Présentation de l'architecture MVC2 **2**

4 Description technique de l'application **3**

4.1 Structure du projet 3

4.2 Couche Modèle 4

4.2.1 Entité Produit 4

4.2.2 DAO Hibernate 5

4.3 Couche Contrôleur 6

4.4 Configuration web.xml 8

4.5 Couche Vue 9

5 Intégration de Hibernate **9**

5.1 Configuration 9

5.2 Gestion des sessions 10

5.3 Base de données 10

6 Comparaison entre les architectures MVC1 et MVC2 **11**

7 Conclusion **12**

1 Introduction

Dans le cadre du module Java EE, le TP4 a permis de développer une application web dynamique de gestion des produits basée sur l'architecture MVC1.

Cette application proposait des fonctionnalités CRUD complètes ainsi que des mécanismes de sécurité tels que l'authentification et la gestion des rôles.

Le TP5 consiste à reprendre ce même projet tout en adoptant l'architecture MVC2 et en intégrant Hibernate afin d'améliorer la séparation des responsabilités et la gestion de la persistance.

2 Objectifs du TP5

- Implémenter l'architecture MVC2
- Structurer l'application en couches
- Intégrer Hibernate (ORM)
- Réaliser les opérations CRUD avec base de données
- Améliorer la maintenabilité du système

3 Présentation de l'architecture MVC2

MVC2 repose sur une séparation claire :

- Modèle : logique métier et données
- Vue : affichage avec JSP
- Contrôleur : gestion des requêtes via Servlets

Flux :

Client → Controller → Model → View

Contrairement à MVC1, cette architecture améliore la modularité et la maintenabilité.

4 Description technique de l'application

4.1 Structure du projet

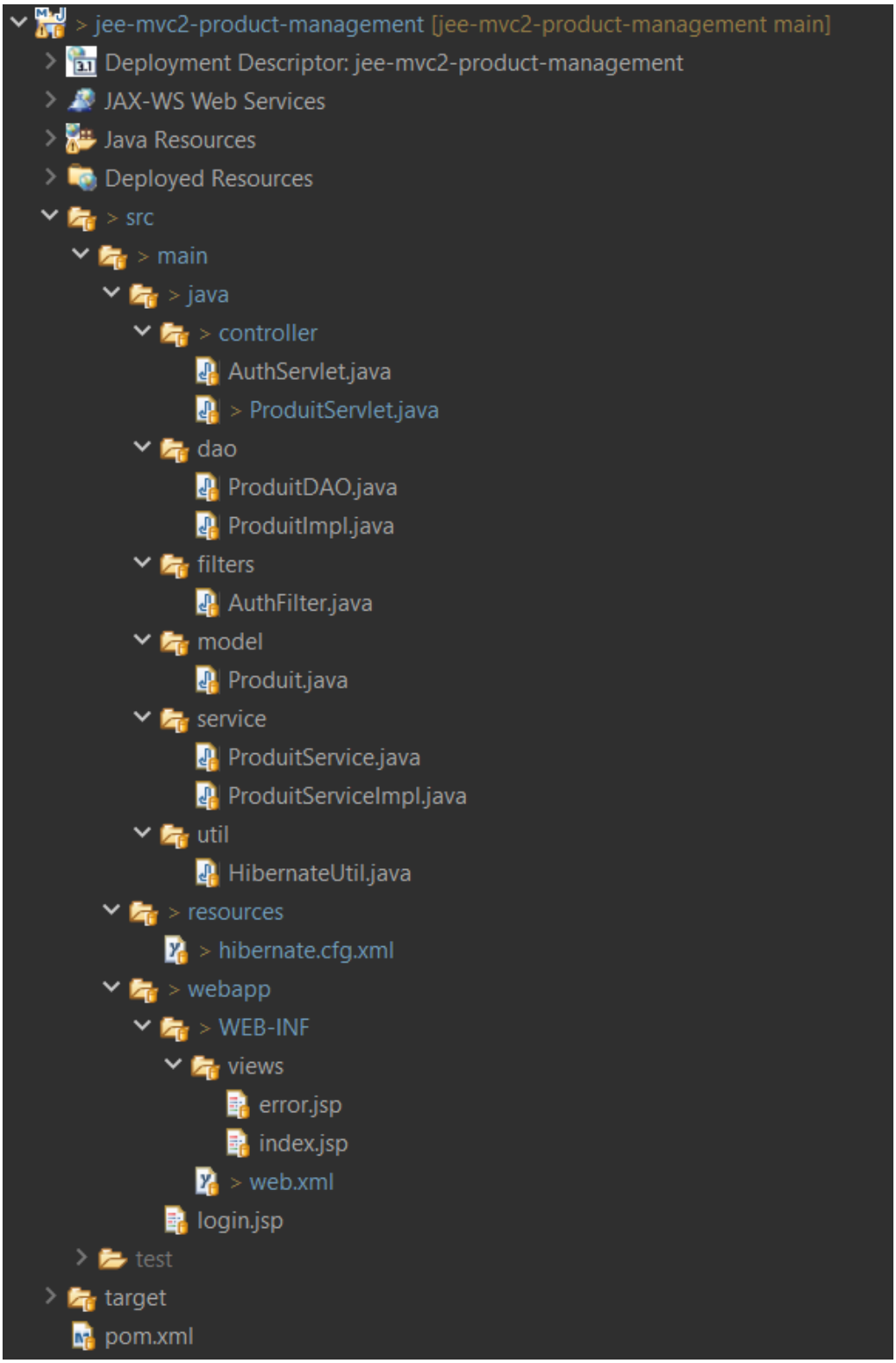

FIGURE 1 – Organisation du projet en packages (model, dao, controller, view)

La structure du projet suit une organisation en couches permettant une séparation claire des responsabilités : le package *model* contient les entités, *dao* gère l'accès aux données, *controller* traite les requêtes, et *view* regroupe les pages JSP.

4.2 Couche Modèle

4.2.1 Entité Produit

```java
package model;

import javax.persistence.*;

@Entity
@Table(name = "produit")
public class Produit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;
    private String description;
    private Double prix;

    public Produit() {}

    public Produit(String nom, String description, Double prix) {
        this.nom = nom;
        this.description = description;
        this.prix = prix;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public Double getPrix() { return prix; }
    public void setPrix(Double prix) { this.prix = prix; }
}
```

FIGURE 2 – Classe Produit annotée avec Hibernate

Cette classe représente l'entité principale du système. Elle est mappée à une table dans la base de données via les annotations Hibernate.

4.2.2 DAO Hibernate

```java
package dao;

import model.Produit;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.util.List;

public class ProduitImpl implements ProduitDAO {

    public void addProduit(Produit p) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();

        session.save(p);

        tx.commit();
        session.close();
    }

    public void deleteProduit(Long id) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();

        Produit p = session.get(Produit.class, id);
        if (p != null) {
            session.delete(p);
        }

        tx.commit();
        session.close();
    }

    public Produit getProduitById(Long id) {
        Session session = HibernateUtil.getSessionFactory().openSession();

        Produit p = session.get(Produit.class, id);

        session.close();
        return p;
    }

    public List<Produit> getAllProduits() {
        Session session = HibernateUtil.getSessionFactory().openSession();

        List<Produit> list = session.createQuery("from Produit", Produit.class).list();

        session.close();
        return list;
    }

    public void updateProduit(Produit p) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();

        session.update(p);

        tx.commit();
        session.close();
    }
}
```

FIGURE 3 – Implémentation DAO utilisant Hibernate

La couche DAO utilise Hibernate pour effectuer les opérations CRUD de manière simplifiée.

4.3 Couche Contrôleur

```java
@WebServlet("/produit")
public class ProduitServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    ProduitServiceImpl service = ProduitServiceImpl.getInstance();
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        HttpSession session = req.getSession(false);
        if (session == null || session.getAttribute("user") == null) {
            resp.sendRedirect(req.getContextPath() + "/login.jsp");
            return;
        }
        String action = req.getParameter("action");
        if (action == null) action = "list";
        switch (action) {
            case "list":
                req.setAttribute("mode", "list");
                req.setAttribute("produits", service.getAllProduits());
                req.getRequestDispatcher("/WEB-INF/views/index.jsp").forward(req, resp);
                break;
            case "addForm":
                req.setAttribute("mode", "form");
                req.setAttribute("formAction", "add");
                req.getRequestDispatcher("/WEB-INF/views/index.jsp").forward(req, resp);
                break;
            case "editForm":
                Long id = Long.parseLong(req.getParameter("id"));

                req.setAttribute("mode", "form");
                req.setAttribute("formAction", "update");
                req.setAttribute("produit", service.getProduitById(id));

                req.getRequestDispatcher("/WEB-INF/views/index.jsp").forward(req, resp);
                break;
            case "delete":
                Long idDel = Long.parseLong(req.getParameter("id"));
                service.deleteProduit(idDel);

                resp.sendRedirect(req.getContextPath() + "/produit?action=list");
                break;
            case "searchById":
                Long id1 = Long.parseLong(req.getParameter("id"));
                Produit p = service.getProduitById(id1);

                req.setAttribute("mode", "list");

                if (p != null) {
                    req.setAttribute("produits", java.util.Arrays.asList(p));
                } else {
                    req.setAttribute("produits", service.getAllProduits());
                    req.setAttribute("error", "Produit introuvable");
                }
                req.getRequestDispatcher("/WEB-INF/views/index.jsp")
                        .forward(req, resp);
                break;
            default:
                resp.sendRedirect(req.getContextPath() + "/produit?action=list");
        }
    }


```

```java

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws IOException {

        String action = req.getParameter("action");
        if ("add".equals(action)) {
            Produit p = new Produit(
                    req.getParameter("nom"),
                    req.getParameter("description"),
                    Double.parseDouble(req.getParameter("prix"))
            );
            service.addProduit(p);
        } else if ("update".equals(action)) {
            Produit p = new Produit(
                    req.getParameter("nom"),
                    req.getParameter("description"),
                    Double.parseDouble(req.getParameter("prix"))
            );
            p.setId(Long.parseLong(req.getParameter("id")));
            service.updateProduit(p);
        }
        resp.sendRedirect(req.getContextPath() + "/produit?action=list");
    }
}
```

FIGURE 4 – Servlet Controller centralisé

Le contrôleur implémente une servlet centralisée chargée de gérer l'ensemble des requêtes HTTP liées à la gestion des produits. Il joue le rôle d'intermédiaire entre la couche de présentation et la couche métier.

Dans la méthode `doGet`, le contrôleur commence par vérifier l'authentification de l'utilisateur via la session. Ensuite, il analyse le paramètre `action` afin de déterminer le traitement à exécuter (affichage de la liste, ajout, modification, suppression ou recherche). Chaque action est associée à une redirection vers la vue appropriée ou à un traitement métier via le service.

La méthode `doPost` est utilisée pour la gestion des opérations d'ajout et de mise à jour des produits. Elle récupère les données envoyées par le formulaire, construit un objet métier `Produit`, puis délègue l'opération correspondante à la couche service.

Ainsi, cette servlet assure la centralisation du contrôle, la gestion du flux applicatif et la séparation des responsabilités conformément au modèle MVC2.

4.4 Configuration web.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
         http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
         version="3.1">

    <display-name>Tp5Project</display-name>
    <welcome-file-list>
        <welcome-file>login.jsp</welcome-file>
    </welcome-file-list>

    <filter>
        <filter-name>AuthFilter</filter-name>
        <filter-class>filters.AuthFilter</filter-class>
    </filter>
    <filter-mapping>
        <filter-name>AuthFilter</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>

    <error-page>
        <error-code>404</error-code>
        <location>/error.jsp</location>
    </error-page>
    <error-page>
        <error-code>500</error-code>
        <location>/error.jsp</location>
    </error-page>
</web-app>
```

FIGURE 5 – Configuration du fichier web.xml

Le fichier web.xml permet de définir les servlets, leurs mappings ainsi que les règles de navigation de l'application.

4.5 Couche Vue

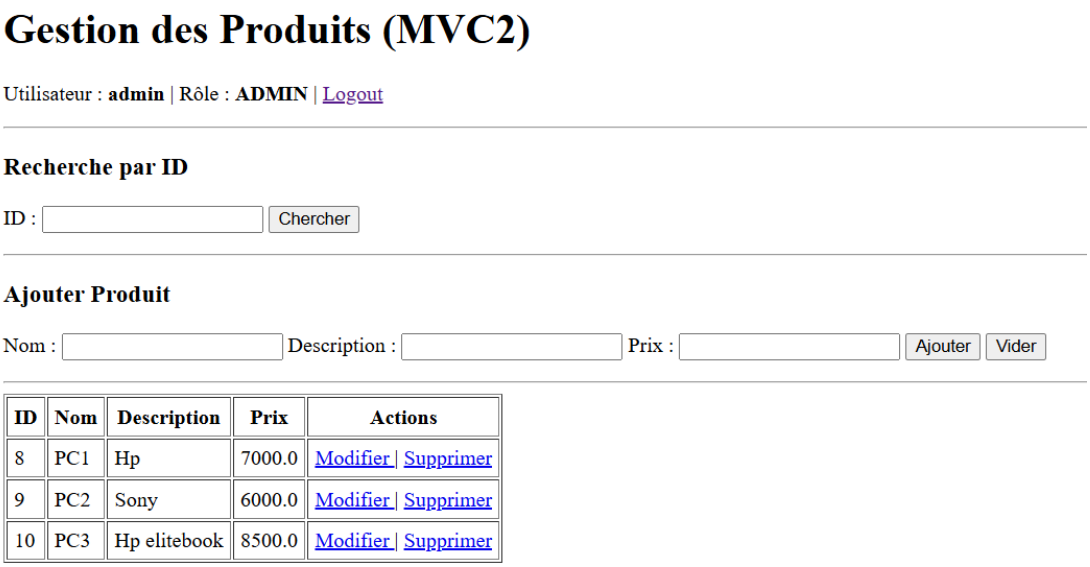

FIGURE 6 – Interface JSP affichant les produits

Les JSP sont utilisées uniquement pour l'affichage, conformément au modèle MVC2.

5 Intégration de Hibernate

5.1 Configuration

```xml
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "https://hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/tp_mvc2</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password"></property>

        <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>
        <property name="hibernate.hbm2ddl.auto">update</property>

        <mapping class="model.Produit"/>
    </session-factory>
</hibernate-configuration>
```

FIGURE 7 – Fichier de configuration Hibernate

Ce fichier contient les paramètres de connexion à la base de données ainsi que les classes mappées.

5.2 Gestion des sessions

```java
package util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static final SessionFactory sessionFactory =
            new Configuration().configure("hibernate.cfg.xml")
                                .buildSessionFactory();

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

FIGURE 8 – Gestion des sessions Hibernate

Hibernate utilise des sessions pour interagir avec la base de données et gérer les transactions.

5.3 Base de données

←T→				id	nom	description	prix
☐	Éditer	Copier	Supprimer	8	PC1	Hp	7000
☐	Éditer	Copier	Supprimer	9	PC2	Sony	6000
☐	Éditer	Copier	Supprimer	10	PC3	Hp elitebook	8500

FIGURE 9 – Table produits dans la base de données

Cette figure montre la table générée automatiquement par Hibernate, contenant les informations des produits.

6 Comparaison entre les architectures MVC1 et MVC2

Afin de mieux comprendre l'apport de l'architecture MVC2, il est pertinent de la comparer avec l'architecture MVC1 utilisée dans le TP4. Cette comparaison met en évidence les améliorations en termes de structuration, de maintenabilité et de gestion des responsabilités.

Critère	**Architecture MVC1**	**Architecture MVC2**
Organisation générale	Faible séparation des responsabilités ; mélange partiel entre la logique métier et la présentation	Séparation claire entre les couches (Modèle, Vue, Contrôleur) garantissant une meilleure modularité
Gestion du contrôleur	Le contrôle est réparti entre les JSP et les Servlets, ce qui peut entraîner une confusion dans le flux de traitement	Le contrôleur est centralisé au niveau des Servlets, assurant une gestion cohérente et structurée des requêtes
Couche Vue	Les JSP contiennent à la fois de la logique de traitement et de l'affichage	Les JSP sont dédiées uniquement à la présentation, sans logique métier
Maintenabilité	Moyenne, en raison du couplage entre les composants	Élevée, grâce à la séparation stricte des responsabilités
Évolutivité	Limitée, difficulté d'ajouter de nouvelles fonctionnalités sans impacter le reste du système	Forte, architecture flexible facilitant les extensions futures
Persistance des données	Implémentation manuelle via DAO (souvent avec JDBC)	Utilisation d'un framework ORM (Hibernate) simplifiant la gestion des données

Cette comparaison montre clairement que l'architecture MVC2 constitue une évolution significative par rapport à MVC1. Elle permet de développer des applications plus robustes, mieux organisées et plus faciles à maintenir, tout en facilitant l'intégration de technologies modernes telles que les frameworks ORM.

7 Conclusion

Ce travail pratique a permis d'implémenter une architecture MVC2, offrant une organisation plus structurée et une meilleure séparation des responsabilités par rapport à l'approche MVC1.

L'intégration de Hibernate a contribué à simplifier la gestion de la persistance des données, tout en améliorant la robustesse, la maintenabilité et la qualité globale de l'application.

Ainsi, cette architecture constitue une base solide pour l'évolution vers des solutions plus avancées et industrielles, notamment les frameworks modernes tels que Spring.